Candidate Job Prediction System Using Decision Tree Dataset: User Interface and ExternalInteraction

Dataset Link: https://www.kaggle.com/datasets/surendra365/recruitement-dataset

Student Name:

Student ID:

Table of Contents

1. Application and Workflow Selection 4

1.1 Introduction and Motivation of the Selected Application 4

1.2 Explanation of the Selected Workflow with Diagram 4

1.3 Objectives of the Selected Intelligent System.....5

1.4 Impact of the Selected Project if Developed.....5

2. Selection of AI Technologies 6

2.1 Identification of Similar Systems 6

2.2 Innovative and Original Approaches Using Existing Literature 6

2.3 Justification for Choosing AI Methods and Technologies 7

3. Simulations and Development Setup 7

3.1 Simulation Environments and Setup Parameters 7

3.2 Agent-wise Task Distribution, Functionality, and Dataset.....8

3.3 Encoding and Data Processing Prior to Simulation 9

4. Use Case Explanation with Evidence 9

4.1 Step-by-Step Workflow Execution with Evidence 9

4.2 Developed User Interface and External Interaction 21

5. Conclusion 22

References 23

List of Figures

Figure 1Workflow Diagram.....5
Figure 2 Install Kaggel (Development Setup).....8
Figure 3 Agent 1 Implementations.....10
Figure 4 Agent 1 Output.....11
Figure 5 Agent 2 Implementations.....12
Figure 6 Agent 2 Implementations 2.....13
Figure 7 Agent 2 Output.....14
Figure 8 Agent 3 Implementations 1.....15
Figure 9 Agent 3 Implementations 2.....16
Figure 10 Agent 3 Output (Decision Tree - Confusion Matrix) 17
Figure 11 Agent 3 Output (Decision Tree - ROC Curve).....18
Figure 12 Agent 3 Output (Random Forest - Confusion Matrix and ROC Curve) 19
Figure 13 Agent 4 Implementations 1.....20
Figure 14 Agent 4 Implementations 2.....20
Figure 15 Agent 4 Output.....21

1. Application and Workflow Selection

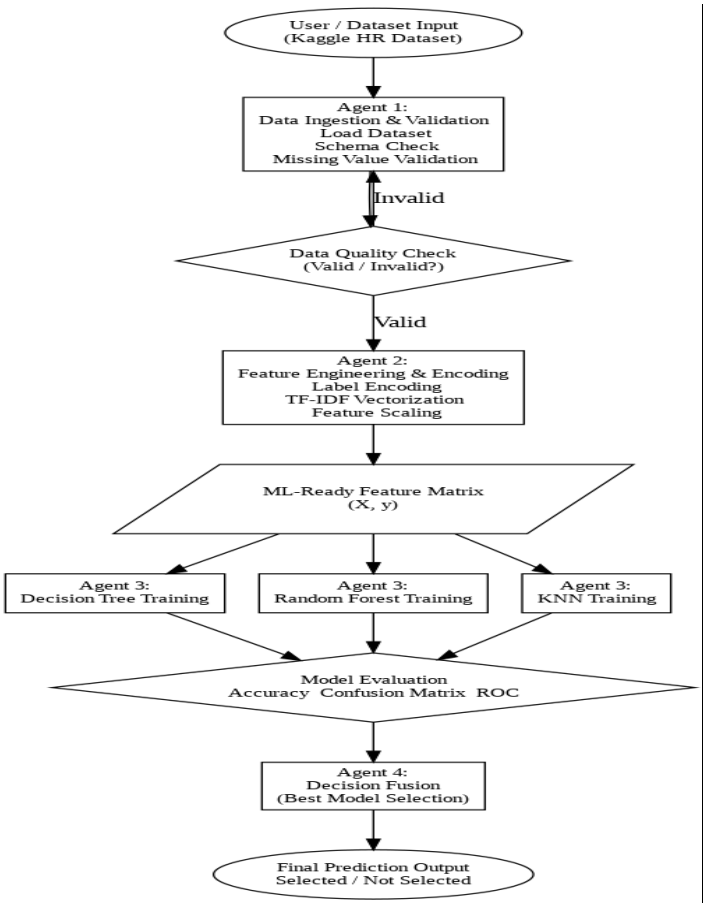
1.1 Introduction and Motivation of the Selected Application

Human Resource (HR) recruitment is an expensive and necessary process in the organization, especially where large number of job applications are involved. The cost of manual screening through resumes is time consuming and highly suspicious to human judgments besides it is not always consistent among recruiters. As more and more digital recruitment platforms become available organizations can now receive thousands of applications to a single job position and therefore traditional recruitment practice is ineffective and prone to error. This is what drives the desire to come up with smart automation that, when done, will help HR teams to shortlist suitable candidates in an objective and efficient manner (Nguyen et al., 2025). The chosen application is based on the automation of the prediction of the suitability of the candidates to the job with the help of an Agentic AI approach. Through utilizing machine learning and smart agents, the system is expected to examine profile of candidates, resumes, and job description to determine whether or not a given candidate is a good fit to a particular job. This will enhance the speed of decision-making, consistency, and assist the HR professionals by saving them the time of doing the job in a manual way and also this kind of approach will maintain transparency in the decision made on the recruitment part.

1.2 Explanation of the Selected Workflow with Diagram

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# Install Kaggle
!pip install -q kaggle

KAGGLE_USERNAME = "your_kaggle_username"
KAGGLE_KEY = "your_kaggle_api_key"

import os

# Set environment variables
os.environ["KAGGLE_USERNAME"] = KAGGLE_USERNAME
os.environ["KAGGLE_KEY"] = KAGGLE_KEY

# Create directory for dataset
os.makedirs("recruitment_dataset", exist_ok=True)

# Download the dataset
!kaggle datasets download -d surendra365/recruitement-dataset -p recruitment_dataset

# Unzip the dataset
!unzip -o recruitment_dataset/recruitement-dataset.zip -d recruitment_dataset

# List files
os.listdir("recruitment_dataset")
```

```
Dataset URL: https://www.kaggle.com/datasets/surendra365/recruitement-dataset
License(s): DbCL-1.0
Downloading recruitement-dataset.zip to recruitment_dataset
  0% 0.00/1.58M [00:00<?, ?B/s]
100% 1.58M/1.58M [00:00<00:00, 237MB/s]
Archive:  recruitment_dataset/recruitement-dataset.zip
  inflating: recruitment_dataset/job_applicant_dataset.csv
['job_applicant_dataset.csv', 'recruitement-dataset.zip']
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

AGENT 1 – Data Ingestion and Validation

```python
# AGENT 1: Data Ingestion & Validation

# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
file_path = "recruitment_dataset/job_applicant_dataset.csv"
df = pd.read_csv(file_path)

# Display first few rows
print(" FIRST 5 ROWS ")
print(df.head())

# Dataset shape
print("\n DATASET SHAPE ")
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")

# Column names
print("\n COLUMN NAMES ")
print(df.columns.tolist())

# Dataset information
print("\n DATASET INFO ")
df.info()

# Missing values check
print("\n MISSING VALUES ")
print(df.isnull().sum())

# Statistical summary
print("\n STATISTICAL SUMMARY ")
print(df.describe())

# Identify target column (assumed last column)
target_column = df.columns[-1]

print("\n TARGET COLUMN ")
print(f"Target Column Name: {target_column}")
```

[redacted]

```
plt.show()
```

```
FIRST 5 ROWS
  Job Applicant Name  Age  Gender             Race   Ethnicity  \
0       Daisuke Mori   29    Male  Mongoloid/Asian  Vietnamese
1     Taichi Shimizu   31    Male  Mongoloid/Asian    Filipino
2       Sarah Martin   46  Female  White/Caucasian       Dutch
3       Keith Hughes   43    Male    Negroid/Black   Caribbean
4        James Davis   49    Male  White/Caucasian     English

                                              Resume             Job Roles  \
0  Proficient in Injury Prevention, Motivation, N...         Fitness Coach
1  Proficient in Healthcare, Pharmacology, Medica...             Physician
2  Proficient in Forecasting, Financial Modelling...     Financial Analyst
3  Proficient in Budgeting, Supply Chain Optimiza...  Supply Chain Manager
4  Proficient in Logistics, Negotiation, Procurem...  Supply Chain Manager

                                     Job Description  Best Match
0   A Fitness Coach is responsible for helping cl...           0
1  Diagnose and treat illnesses, prescribe medica...           0
2  As a Financial Analyst, you will be responsibl...           0
3  A Supply Chain Manager oversees the entire sup...           1
4  A Supply Chain Manager oversees the entire sup...           1

DATASET SHAPE
Rows: 10000, Columns: 9

COLUMN NAMES
['Job Applicant Name', 'Age', 'Gender', 'Race', 'Ethnicity', 'Resume', 'Job Roles', 'Job Description', 'Best Match']

DATASET INFO
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Job Applicant Name  10000 non-null  object
 1   Age                 10000 non-null  int64
 2   Gender              10000 non-null  object
 3   Race                10000 non-null  object
 4   Ethnicity           10000 non-null  object
 5   Resume              10000 non-null  object
 6   Job Roles           10000 non-null  object
 7   Job Description     10000 non-null  object
 8   Best Match          10000 non-null  int64
dtypes: int64(2), object(7)
memory usage: 703.3+ KB
```

[redacted]

[redacted]

[redacted]

AGENT 2 – Feature Engineering and Encoding Agent

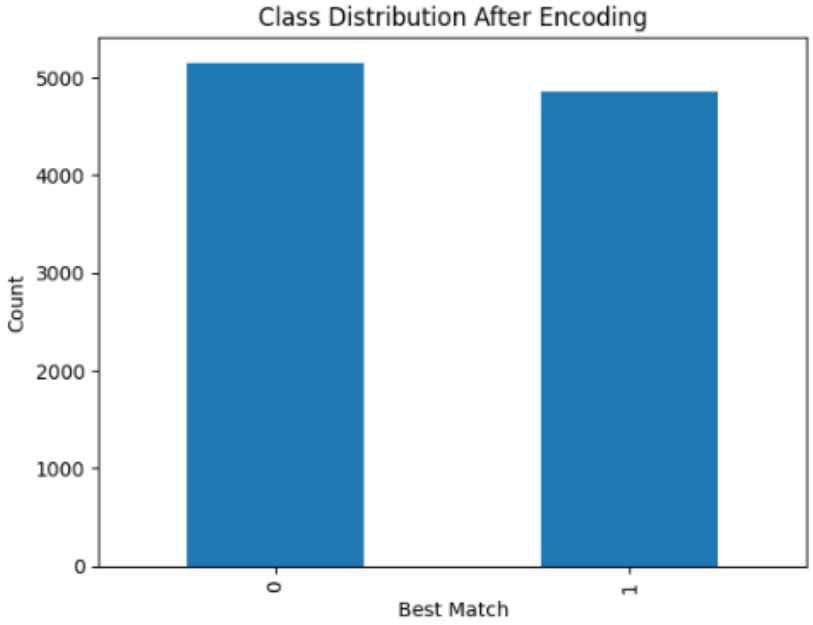
```python
# AGENT 2: Feature Engineering & Encoding

from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from scipy.sparse import hstack

# Copy dataset to avoid overwriting
data = df.copy()

print(" ORIGINAL DATA SHAPE ")
print(data.shape)


# Encode categorical variables

label_encoders = {}
categorical_cols = ['Gender', 'Race', 'Ethnicity', 'Job Roles']

for col in categorical_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le

print("\nEncoded categorical columns:")
print(categorical_cols)


# TF-IDF Vectorization for text

tfidf_resume = TfidfVectorizer(max_features=300)
tfidf_job_desc = TfidfVectorizer(max_features=300)

resume_features = tfidf_resume.fit_transform(data['Resume'])
job_desc_features = tfidf_job_desc.fit_transform(data['Job Description'])

print("\nTF-IDF Resume Shape:", resume_features.shape)
print("TF-IDF Job Description Shape:", job_desc_features.shape)
```

[redacted]

```python
# Numerical features

numerical_features = data[['Age', 'Gender', 'Race', 'Ethnicity', 'Job Roles']]

scaler = StandardScaler()
numerical_scaled = scaler.fit_transform(numerical_features)

print("\nNumerical Features Shape:", numerical_scaled.shape)


# Combine all features

X = hstack([numerical_scaled, resume_features, job_desc_features])
y = data['Best Match']

print("\nFinal Feature Matrix Shape:", X.shape)
print("Target Vector Shape:", y.shape)


# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\n TRAIN-TEST SPLIT ")
print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])


# Class Distribution Visualization

plt.figure()
y.value_counts().plot(kind='bar')
plt.title("Class Distribution After Encoding")
plt.xlabel("Best Match")
plt.ylabel("Count")
plt.show()
```

[redacted]

```
 ORIGINAL DATA SHAPE
(10000, 9)

Encoded categorical columns:
['Gender', 'Race', 'Ethnicity', 'Job Roles']

TF-IDF Resume Shape: (10000, 300)
TF-IDF Job Description Shape: (10000, 300)

Numerical Features Shape: (10000, 5)

Final Feature Matrix Shape: (10000, 605)
Target Vector Shape: (10000,)

 TRAIN-TEST SPLIT
Training Samples: 8000
Testing Samples: 2000
```



[redacted]

[redacted]

[redacted]

AGENT 3 – Model Training and Evaluation

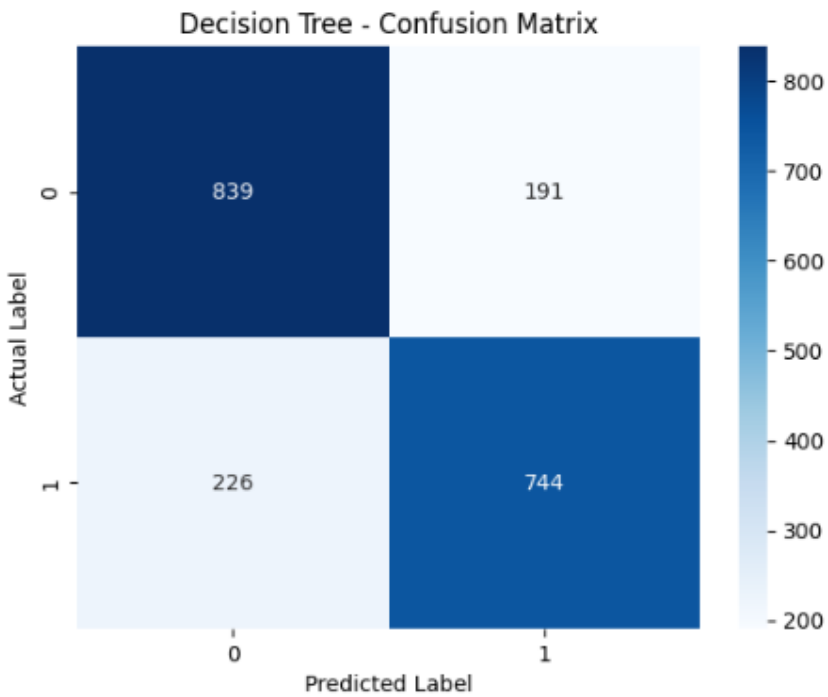
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_curve,
    auc
)
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns


# Initialize Models

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}

accuracy_results = {}


# Train, Evaluate, and Visualize

for model_name, model in models.items():
    print(f"\n====== {model_name} =======")

    # Train model
    model.fit(X_train, y_train)

    # Predictions
    y_pred = model.predict(X_test)

    # Accuracy
    acc = accuracy_score(y_test, y_pred)
```

[redacted]

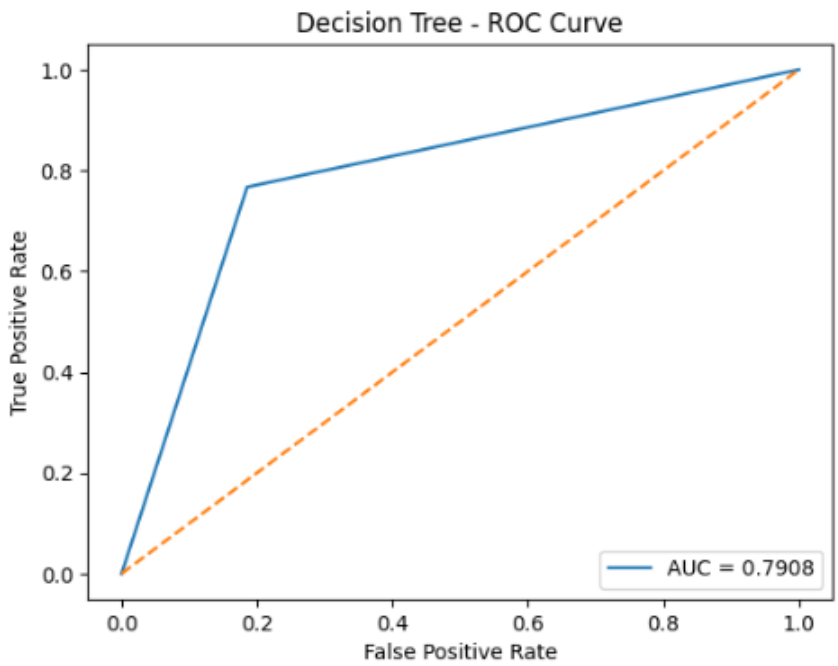
```python
accuracy_results[model_name] = acc
print(f"Accuracy: {acc:.4f}")

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))


# Confusion Matrix Plot

cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f"{model_name} - Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()


# ROC Curve & AUC

if hasattr(model, "predict_proba"):
    y_prob = model.predict_proba(X_test)[:, 1]
else:
    y_prob = model.predict(X_test)

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}")
plt.plot([0, 1], [0, 1], linestyle='--')
plt.title(f"{model_name} - ROC Curve")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.show()
```

[redacted]

```
...
====== Decision Tree =======
Accuracy: 0.7915

Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.81      0.80      1030
           1       0.80      0.77      0.78       970

    accuracy                           0.79      2000
   macro avg       0.79      0.79      0.79      2000
weighted avg       0.79      0.79      0.79      2000
```

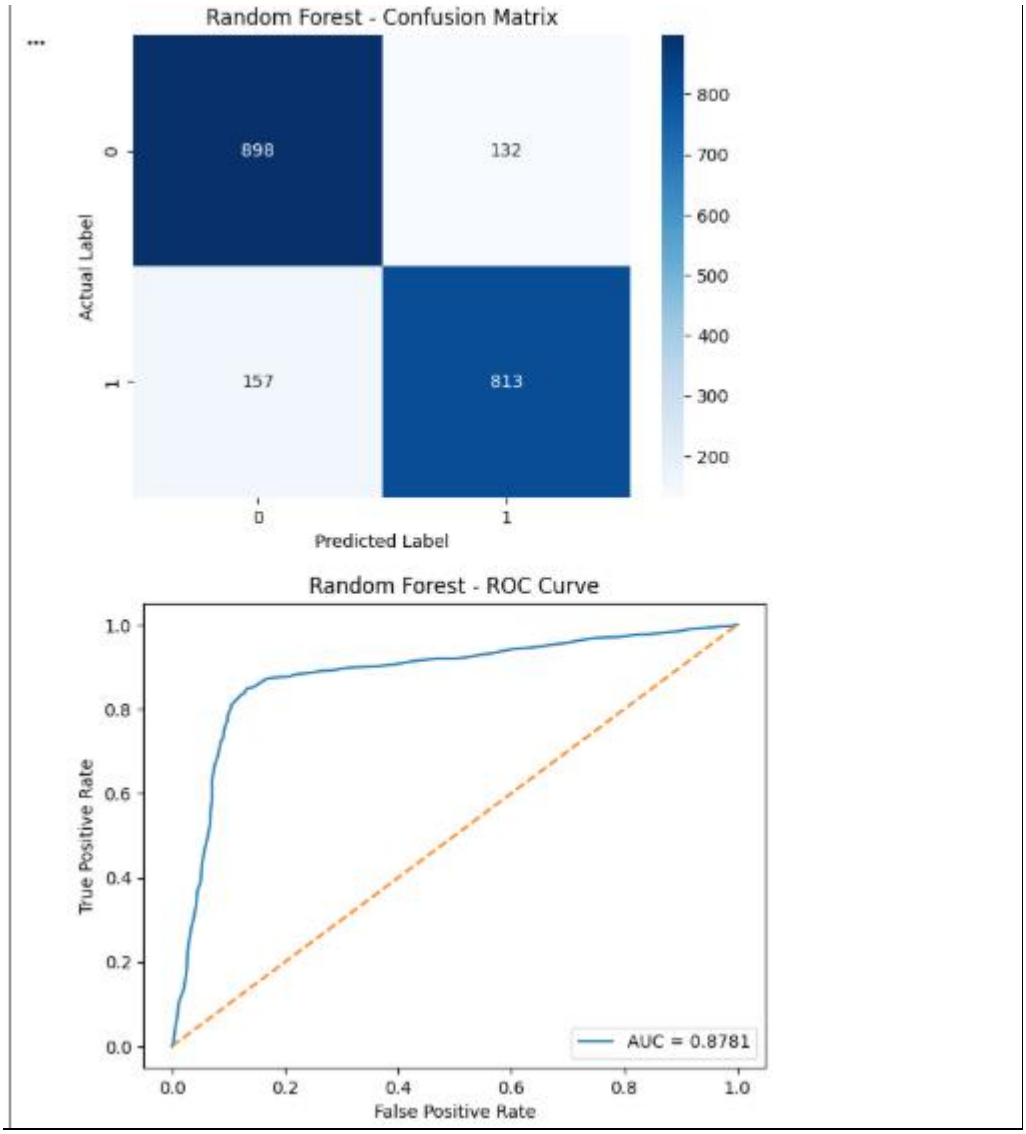


[redacted]




```
====== Random Forest =======
Accuracy: 0.8555

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.87      0.86      1030
           1       0.86      0.84      0.85       970

    accuracy                           0.86      2000
   macro avg       0.86      0.85      0.86      2000
weighted avg       0.86      0.86      0.86      2000
```

[redacted]


Random Forest - Confusion Matrix
898
132
157
813
Actual Label
Predicted Label
0
1
800
700
600
500
400
300
200
Random Forest - ROC Curve
True Positive Rate
False Positive Rate
AUC = 0.8781
0.0
0.2
0.4
0.6
0.8
1.0

[redacted]

AGENT 4: Decision Fusion and Final Prediction Agent

```python
# Identify best model based on accuracy
best_model_name = max(accuracy_results, key=accuracy_results.get)
best_model = models[best_model_name]

print(" DECISION FUSION AGENT ")
print(f"Best Performing Model Selected: {best_model_name}")
print(f"Accuracy: {accuracy_results[best_model_name]:.4f}")


# Sample Candidate Input

sample_candidate = {
    'Age': 35,
    'Gender': 'Male',
    'Race': 'White/Caucasian',
    'Ethnicity': 'English',
    'Job Roles': 'Supply Chain Manager',
    'Resume': 'Experience in logistics, inventory control, vendor management and procurement',
    'Job Description': 'Responsible for managing supply chain operations, logistics and procurement'
}

print("\n SAMPLE CANDIDATE INPUT ")
for k, v in sample_candidate.items():
    print(f"{k}: {v}")


# Encode Categorical Inputs

encoded_candidate = [
    sample_candidate['Age'],
    label_encoders['Gender'].transform([sample_candidate['Gender']])[0],
    label_encoders['Race'].transform([sample_candidate['Race']])[0],
    label_encoders['Ethnicity'].transform([sample_candidate['Ethnicity']])[0],
    label_encoders['Job Roles'].transform([sample_candidate['Job Roles']])[0]
]
```

[redacted]

```python
# TF-IDF Transformation

resume_vec = tfidf_resume.transform([sample_candidate['Resume']])
job_desc_vec = tfidf_job_desc.transform([sample_candidate['Job Description']])


# Combine Features

final_input = hstack([encoded_candidate, resume_vec, job_desc_vec])


# Prediction

prediction = best_model.predict(final_input)[0]
probability = best_model.predict_proba(final_input)[0][1]

print("\n FINAL PREDICTION ")
print("Predicted Best Match:", prediction)
print(f"Selection Probability: {probability:.2f}")

if prediction == 1:
    print("Decision: Candidate is SUITABLE for the job")
else:
    print("Decision: Candidate is NOT SUITABLE for the job")
```

[redacted]

```
•••  DECISION FUSION AGENT
Best Performing Model Selected: KNN
Accuracy: 0.8735

 SAMPLE CANDIDATE INPUT
Age: 35
Gender: Male
Race: White/Caucasian
Ethnicity: English
Job Roles: Supply Chain Manager
Resume: Experience in logistics, inventory control, vendor management and procurement
Job Description: Responsible for managing supply chain operations, logistics and procurement

 FINAL PREDICTION
Predicted Best Match: 1
Selection Probability: 1.00
Decision: Candidate is SUITABLE for the job
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
  warnings.warn(
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]